

Strukturni paterni u StudyBuddy sistemu

1. Uvod

StudyBuddy sistem omogućava korisnicima kreiranje sesija učenja, prijavljivanje na sesije, evidentiranje prisustva, pregled obavještenja i upravljanje osnovnim podacima sistema. U prethodnim projektnim zadacima kreiran je osnovni klasni dijagram koji prikazuje glavne klase sistema, kao što su **Korisnik**, **SesijaUcenja**, **PrijavaNaSesiju**, **Prisustvo**, **Predmet**, **Lokacija**, **Uloga** i **Obavjestenje**.

Kako sistem postaje složeniji, javlja se potreba za boljom organizacijom odnosa između klasa. Strukturalni paterni omogućavaju da se klase povežu na pregledniji način, da se smanji međusobna zavisnost i da se sistem lakše proširuje u budućnosti. U StudyBuddy sistemu odabrana su dva strukturalna paterna: **Facade** i **Adapter**.

2. Potreba za dodavanje strukturalnih paterna

U postojećem sistemu određene funkcionalnosti uključuju više povezanih klasa. Na primjer, funkcionalnost prijavljivanja korisnika na sesiju ne koristi samo klasu **PrijavaNaSesiju**, nego uključuje i klasu **Korisnik**, klasu **SesijaUcenja**, provjeru broja slobodnih mjesta i kreiranje obavještenja. Ukoliko bi svaki dio sistema direktno komunicirao sa svim ovim klasama, dijagram bi postao složeniji, a sistem teži za održavanje.

Zbog toga se uvodi **Facade patern**, koji omogućava jednostavniji pristup složenom podsistemu sesija učenja. Također, sistem ima klasu **Obavjestenje**, ali u praksi slanje obavještenja može zavisiti od eksternog email servisa. Kako se postojeće klase ne bi morale prilagođavati vanjskom servisu, uvodi se **Adapter patern**.

Na ovaj način se postojeći sistem ne mijenja u osnovnoj strukturi, nego se nadograđuje dodatnim klasama koje poboljšavaju njegovu fleksibilnost i preglednost.

3. Facade patern

Facade patern se koristi kada je potrebno pojednostaviti pristup složenom podsistemu. Umjesto da klijent direktno koristi više različitih klasa, uvodi se jedna fasadna klasa koja objedinjuje potrebne operacije.

U StudyBuddy sistemu ovaj patern je primijenjen kroz klasu **SesijaFacade**. Ova klasa predstavlja jednostavan ulaz u funkcionalnosti vezane za sesije učenja. Ona koristi postojeće klase sistema, ali sakriva složenost njihove međusobne komunikacije.

Klasa **SesijaFacade** koristi sljedeće klase:

- **Korisnik**
- **SesijaUcenja**
- **PrijavaNaSesiju**
- **Predmet**
- **Lokacija**

- **Obavjestenje**
- **IObavjestenjeSender**

U dijagramu su u klasi **SesijaFacade** definisane metode:

- Metoda **kreirajSesiju** omogućava kreiranje nove sesije učenja na osnovu korisnika koji je kreira, odabranog predmeta i lokacije
- Metoda **prijaviKorisnikaNaSesiju** objedinjuje proces prijave korisnika na sesiju
- Metoda **provjeriSlobodnaMjesta** provjerava da li je moguće dodati novog učesnika.
- Metoda **otkaziSesiju** omogućava otkazivanje postojeće sesije.

Prednost ovog paterna je u tome što se složen proces rada sa sesijama svodi na korištenje jedne klase. Time se smanjuje direktna zavisnost drugih dijelova sistema od velikog broja model klasa.

4. Adapter patern

Adapter patern se koristi kada postojeći sistem treba koristiti klasu ili servis sa drugačijim interfejsom. Adapter služi kao posrednik između sistema i klase koja se ne može direktno koristiti bez prilagođavanja.

U StudyBuddy sistemu ovaj patern je primijenjen na dio sistema koji se odnosi na obavještenja. Postojeća klasa **Obavjestenje** čuva podatke o obavještenju, kao što su naslov, sadržaj, datum slanja, korisnik i tip obavještenja. Međutim, stvarno slanje obavještenja može se izvršavati preko eksternog email servisa.

Problem je u tome što eksterni email servis može imati drugačiju metodu za slanje poruke. Na primjer, u dijagramu klasa **EksterniEmailServis** posjeduje metodu:

- `+sendEmail(primalac : string, naslov : string, sadrzaj : string) : void`

S druge strane, StudyBuddy sistem radi sa objektom klase **Obavjestenje**. Zbog toga je uveden interfejs **IObavjestenjeSender** koji definiše metodu:

- `+posalji(obavjestenje : Obavjestenje) : void`

Klasa **EmailSenderAdapter** implementira interfejs **IObavjestenjeSender** i prilagođava objekat klase **Obavjestenje** formatu koji očekuje eksterni email servis. Ona sadrži atribut:

- `-emailServis : EksterniEmailServis`

i metodu:

- `+posalji(obavjestenje : Obavjestenje) : void`

Na taj način ostatak sistema ne mora znati detalje rada eksternog email servisa. Sistem koristi interfejs **IObavjestenjeSender**, a adapter preuzima odgovornost za prilagođavanje podataka.

Ova struktura omogućava da se u budućnosti doda novi način slanja obavještenja, na primjer SMS ili push notifikacije, bez izmjene postojećih model klasa.

5. Veza između odabranih paterna

U kreiranom dijagramu oba paterna su međusobno povezana. Klasa **SesijaFacade** koristi interfejs **IObavjestenjeSender** kako bi mogla poslati obavještenje nakon kreiranja sesije, prijave korisnika ili otkazivanja sesije.

Time se postiže bolja organizacija sistema:

- **Facade** pojednostavljuje rad sa procesima vezanim za sesije.
- **Adapter** omogućava povezivanje sistema sa eksternim servisom za slanje email obavještenja.
- **SesijaFacade** ne zavisi direktno od klase **EksterniEmailServis**, nego od interfejsa **IObavjestenjeSender**.

Ovakvo rješenje povećava fleksibilnost sistema i omogućava jednostavnije proširenje u budućnosti.

6. Zaključak

Dodavanjem **Facade** i **Adapter** paterna StudyBuddy sistem postaje pregledniji, fleksibilniji i lakši za održavanje. Facade patern pojednostavljuje rad sa složenim procesima koji uključuju sesije učenja, korisnike, prijave i obavještenja. Adapter patern omogućava povezivanje sistema sa eksternim email servisom bez mijenjanja postojećih klasa.

Ovi paterni ne mijenjaju osnovni model sistema, nego ga nadograđuju dodatnim klasama koje poboljšavaju strukturu i pripremaju sistem za buduća proširenja. Zbog toga su **Facade** i **Adapter** prikladni strukturalni paterni za StudyBuddy sistem.